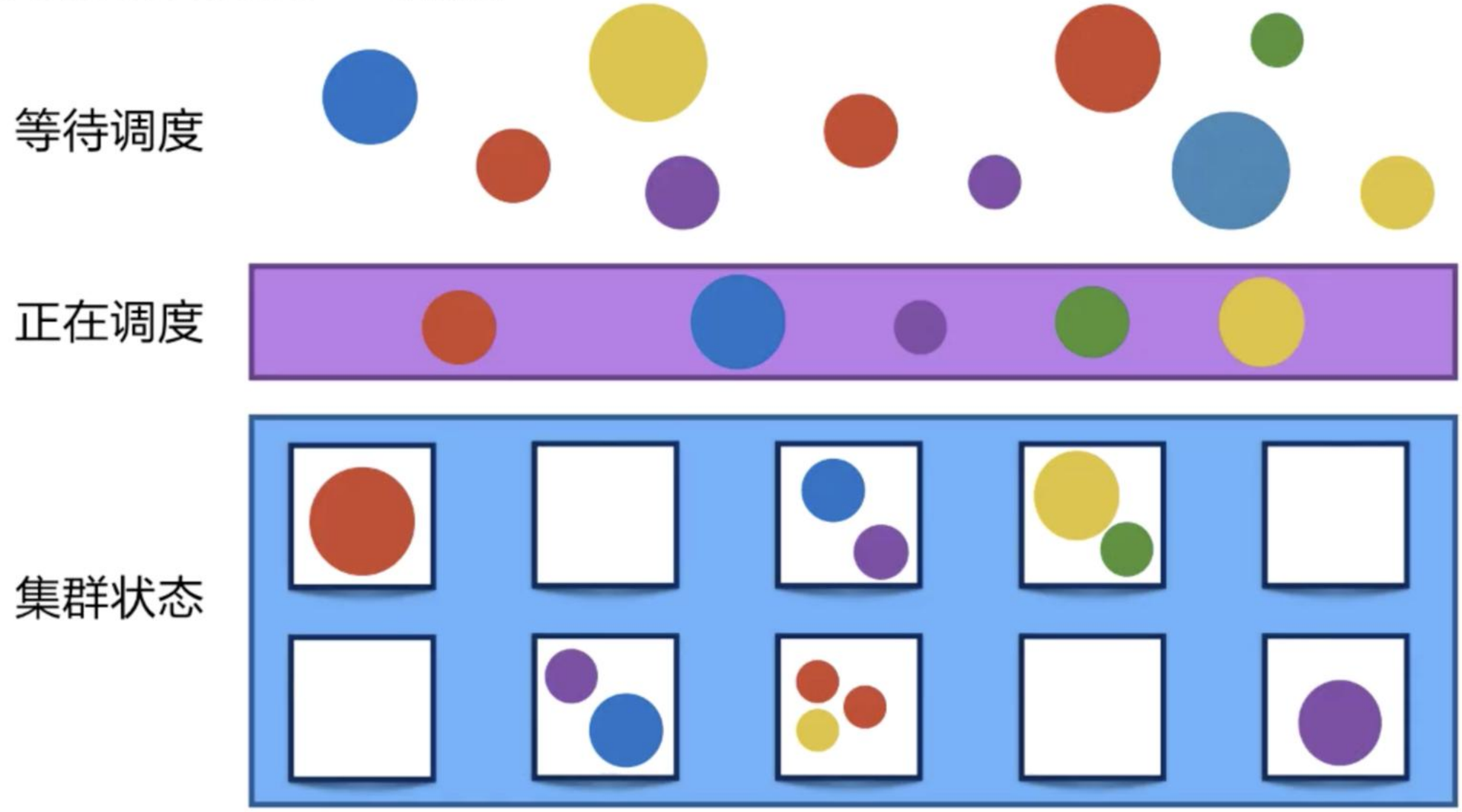
Kubernetes – 调度

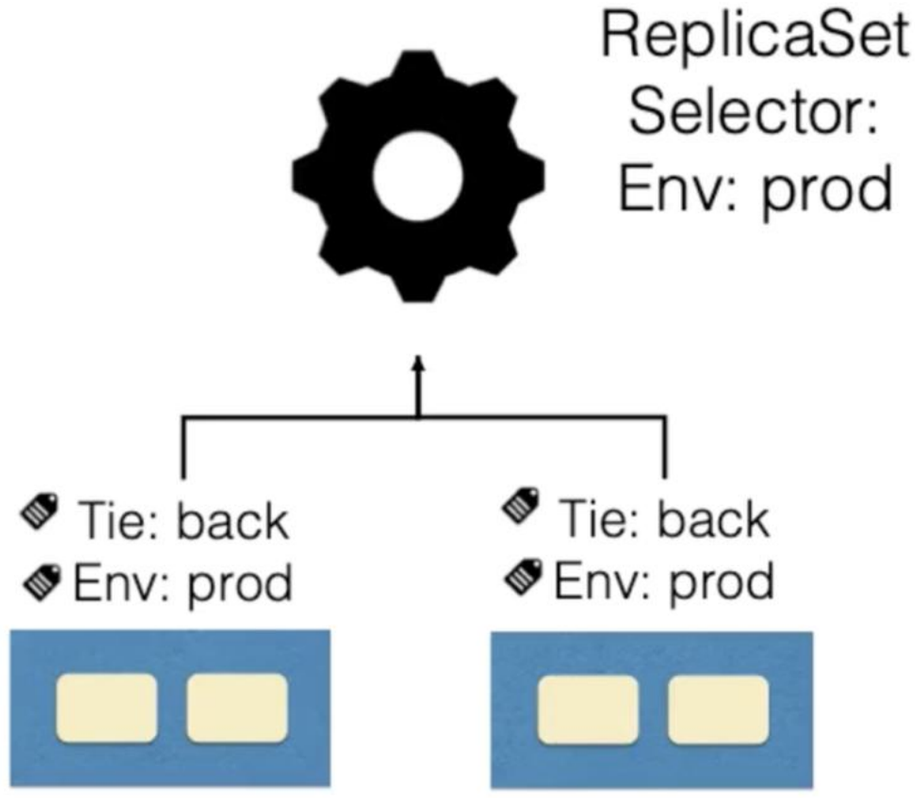




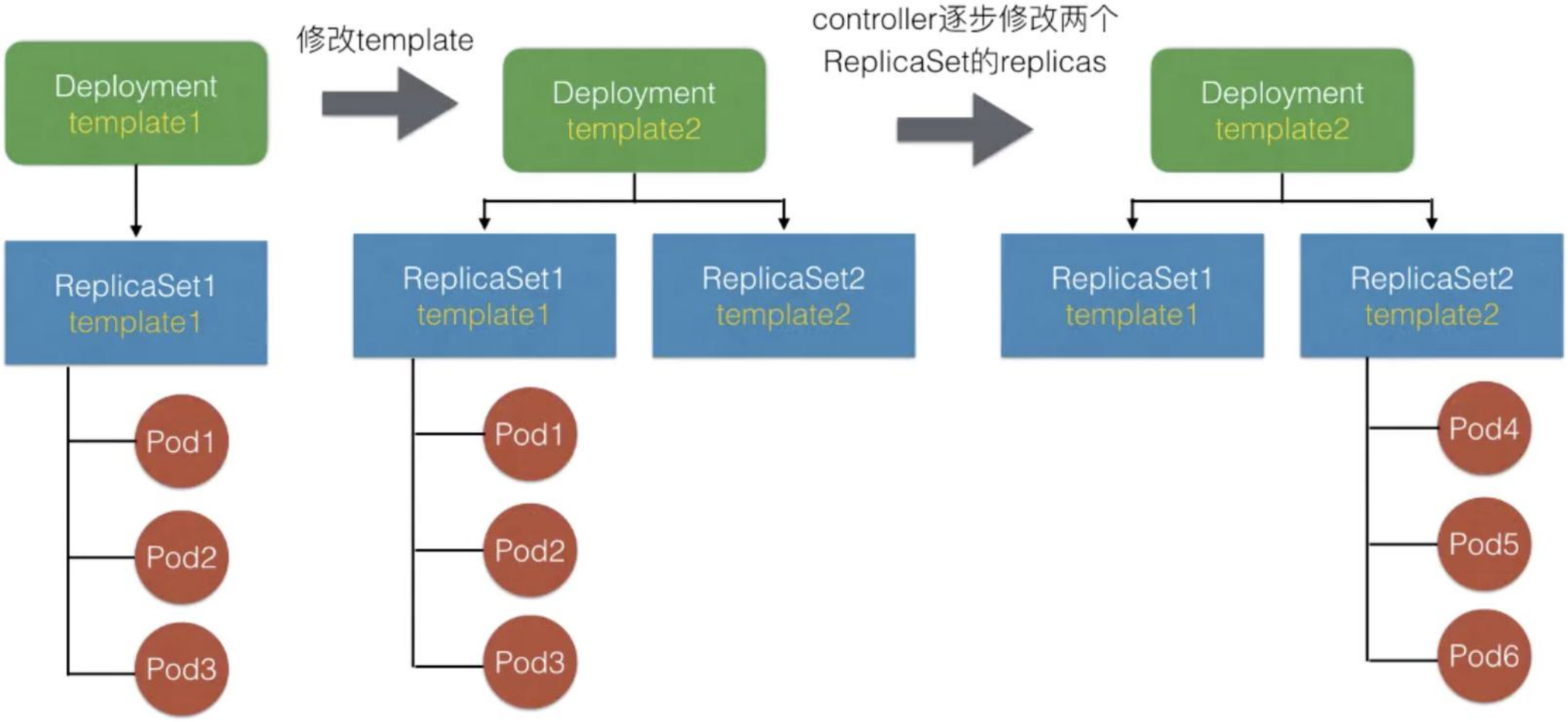
控制循环例子-扩容

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: rsA
  namespace: nsA
spec:
  replicas: 2
  selector:
    matchLabels:
      env: prod
  template:
    metadata:
      labels:
        env: prod
    spec:
      containers:
      - image: registry.cn-hangzhou.aliyuncs.com/acs/nginx
        name: nginx
status:
  replicas: 2
```

2 -> 3



发布模拟


Deployment
template1
ReplicaSet1
template1
Pod1
Pod2
Pod3
修改template
Deployment
template2
ReplicaSet1
template1
ReplicaSet2
template2
Pod1
Pod2
Pod3
controller逐步修改两个
ReplicaSet的replicas
Deployment
template2
ReplicaSet1
template1
ReplicaSet2
template2
Pod4
Pod5
Pod6

spec字段解析

MinReadySeconds：
判断Pod available的最小ready时间

revisionHistoryLimit：
保留历史revision(ReplicaSet)的数量，默认值为10

paused：
标识Deployment只做数量维持、不做新的发布

progressDeadlineSeconds：
判断Deployment status condition为failed的最大时间

```
// Minimum number of seconds for which a newly created pod should be ready
// without any of its container crashing, for it to be considered available.
// Defaults to 0 (pod will be considered available as soon as it is ready)
// +optional
MinReadySeconds int32 `json:"minReadySeconds,omitempty" protobuf:"varint,5,opt,name=minReadySecon

// The number of old ReplicaSets to retain to allow rollback.
// This is a pointer to distinguish between explicit zero and not specified.
// Defaults to 10.
// +optional
RevisionHistoryLimit *int32 `json:"revisionHistoryLimit,omitempty" protobuf:"varint,6,opt,name=re

// Indicates that the deployment is paused.
// +optional
Paused bool `json:"paused,omitempty" protobuf:"varint,7,opt,name=paused"`

// The maximum time in seconds for a deployment to make progress before it
// is considered to be failed. The deployment controller will continue to
// process failed deployments and a condition with a ProgressDeadlineExceeded
// reason will be surfaced in the deployment status. Note that progress will
// not be estimated during the time a deployment is paused. Defaults to 600s.
ProgressDeadlineSeconds *int32 `json:"progressDeadlineSeconds,omitempty" protobuf:"varint,9,opt,n
```



ConfigMap使用

ConfigMap主要被Pod使用，一般用于挂载Pod用的配置文件，环境变量，命令行参数等。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: cm-env-test
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      command: [ "/bin/sh", "-c", "env" ]
      env:
        # 用special-config中的special.how定义环境变量
        - name: SPECIAL_LEVEL_KEY
          valueFrom:
            configMapKeyRef:
              name: special-config
              key: special.how
  restartPolicy: Never
```

用ConfigMap配置环境变量

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: cm-cmd-test
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      command: [ "/bin/sh", "-c", "echo $(SPECIAL_LEVEL_KEY)" ]
      env:
        - name: SPECIAL_LEVEL_KEY
          valueFrom:
            configMapKeyRef:
              name: special-config
              key: SPECIAL_LEVEL
  restartPolicy: Never
```

在命令行参数中使用ConfigMap定义的环境变量

用ConfigMap配置管控命令行参数

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: cm-volume-test
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      command: [ "/bin/sh", "-c", "ls /etc/config/" ]
      volumeMounts:
      - name: config-volume
        mountPath: /etc/config
  volumes:
    - name: config-volume
      configMap:
        name: special-config
  restartPolicy: Never
```

ConfigMap中指定的内容以文件形式挂载在容器中的/etc/config目录下

用ConfigMap挂载配置文件

Secret使用

Secret主要被Pod使用，一般通过volume挂载到指定容器目录，供容器中业务使用。另外在需要访问私有镜像仓库时，也可以引用Secret来实现。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
      readOnly: true
  volumes:
  - name: foo
    secret:
      secretName: mysecret
```

secret以文件形式挂载在/etc/foo下

指定要挂载的secret

Secert挂载在用户指定目录下

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-9f7575d75-mdfp6
  namespace: default
spec:
  containers:
  - image: nginx:1-alpine
    name: nginx
    volumeMounts:
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
      name: default-token-6fn7q
      readOnly: true
  serviceAccount: default
  serviceAccountName: default
  volumes:
  - name: default-token-6fn7q
    secret:
      defaultMode: 420
      secretName: default-token-6fn7q
```

固定挂载到容器中的/var/run/secrets/kubernetes.io/serviceaccount目录下

使用namespace默认的serviceaccount

挂载namespace默认的secret

ServiceAccount用的secret自动挂载在固定目录下(生成ca.crt和token两个文件)



使用私有镜像库

私有镜像仓库的信息可以通过Secret(kubernetes.io/dockerconfigjson)存储在集群中，Pod如果需要使用私有镜像仓库，可以通过如下两种方式来配置:

1. Pod.spec.imagePullSecrets来指定secret
2. ServiceAccount中设置imagePullSecrets，然后自动为使用该SA的Pod注入imagePullSecrets信息。

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: private-reg
spec:
  containers:
  - name: private-reg-container
    image: <your-private-image>
  imagePullSecrets:
  - name: regcred
```

指定imagePullSecret

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  creationTimestamp: 2015-08-07T22:02:39Z
  name: default
  namespace: default
  selfLink: /api/v1/namespaces/default/serviceaccounts/default
  uid: 052fb0f4-3d50-11e5-b066-42010af0d7b6
secrets:
- name: default-token-uudge
imagePullSecrets:
- name: myregistrykey
```

自动为使用该ServiceAccount的Pod注入ImagePullSecrets信息

ServiceAccount设置，Pod创建时自动注入imagePullSecrets

ServiceAccount介绍

ServiceAccount主要用于解决Pod 在集群中的身份认证问题。其中认证使用的授权信息，则利用前面讲到Secret(type=kubernetes.io/service-account-token)进行管理.

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  creationTimestamp: 2019-06-03T08:32:59Z
  name: default
  namespace: default
  resourceVersion: "419220122"
  selfLink: /api/v1/namespaces/default/serviceaccounts/default
  uid: 320061cb-85da-11e9-909b-06b1d3b258c8
secrets:
- name: default-token-6fn7q
```

指定后台的Secret

SeviceAccount定义

```yaml
apiVersion: v1
data:
  ca.crt: LS0tLS1...
  namespace: ZGVmYXVsdA==
  token: ZXlKaGJHY2...
kind: Secret
metadata:
  annotations:
    kubernetes.io/service-account.name: default
    kubernetes.io/service-account.uid: 320061cb-85da-11e9-909b-06b1d3b258c8
  creationTimestamp: 2019-06-03T08:32:59Z
  name: default-token-6fn7q
  namespace: default
  resourceVersion: "419220120"
  selfLink: /api/v1/namespaces/default/secrets/default-token-6fn7q
  uid: 320182a6-85da-11e9-909b-06b1d3b258c8
type: kubernetes.io/service-account-token
```

用于校验服务端的证书信息，base-64编码

Pod的身份认证用的Token，base-64编码

关联的SeriviceAccount信息

Secret类型

ServiceAccount对应的Secret

*Secret由ServiceAccount Controller自动创建



阿里云 CLOUD NATIVE COMPUTING FOUNDATION

如何满足Pod与Pod关系要求？ – Pod亲和调度

- Pod亲和调度 *PodAffinity*
  - 必须和某些Pod调度到一起

    requiredDuringSchedulingIgnoredDuringExecution
  - 优先和某些Pod调度到一起

    preferredDuringSchedulingIgnoredDuringExecution
- Pod反亲和调度 *PodAntiAffinity*
  - 禁止和某些Pod调度到一起

    requiredDuringSchedulingIgnoredDuringExecution
  - 优先不和某些Pod调度到一起

    preferredDuringSchedulingIgnoredDuringExecution
- Operator
  - In/NotIn/Exists/DoesNotExist

优先调度到带了k2=v2标签的Pod所在节点：

```yaml
apiVersion: v1
kind: Pod
metadata:
  namespace: demo-ns
  name: demo-pod
spec:
  containers:
  - image: nginx:latest
    name: demo-container
  affinity:
    podAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchExpressions:
            - key: k2
              operator: In
              values:
              - v2
          topologyKey: "kubernetes.io/hostname"
```

如何做到集群资源合理利用？ – 优先级调度配置

- 怎么使用?
  - 创建PriorityClass
  - 为各个Pod配置上不同的priorityClassName
- 一些内置优先级
  - 内置默认优先级

    DefaultPriorityWhenNoDefaultClassExists=0
  - 用户可配置的最大优先级限制

    HighestUserDefinablePriority= 1000000000
  - 系统级别优先级 SystemCriticalPriority=2000000000
  - 内置系统级别优先级
    - system-cluster-critical
    - system-node-critical

创建一个名为high的priorityClass，得分为10000：

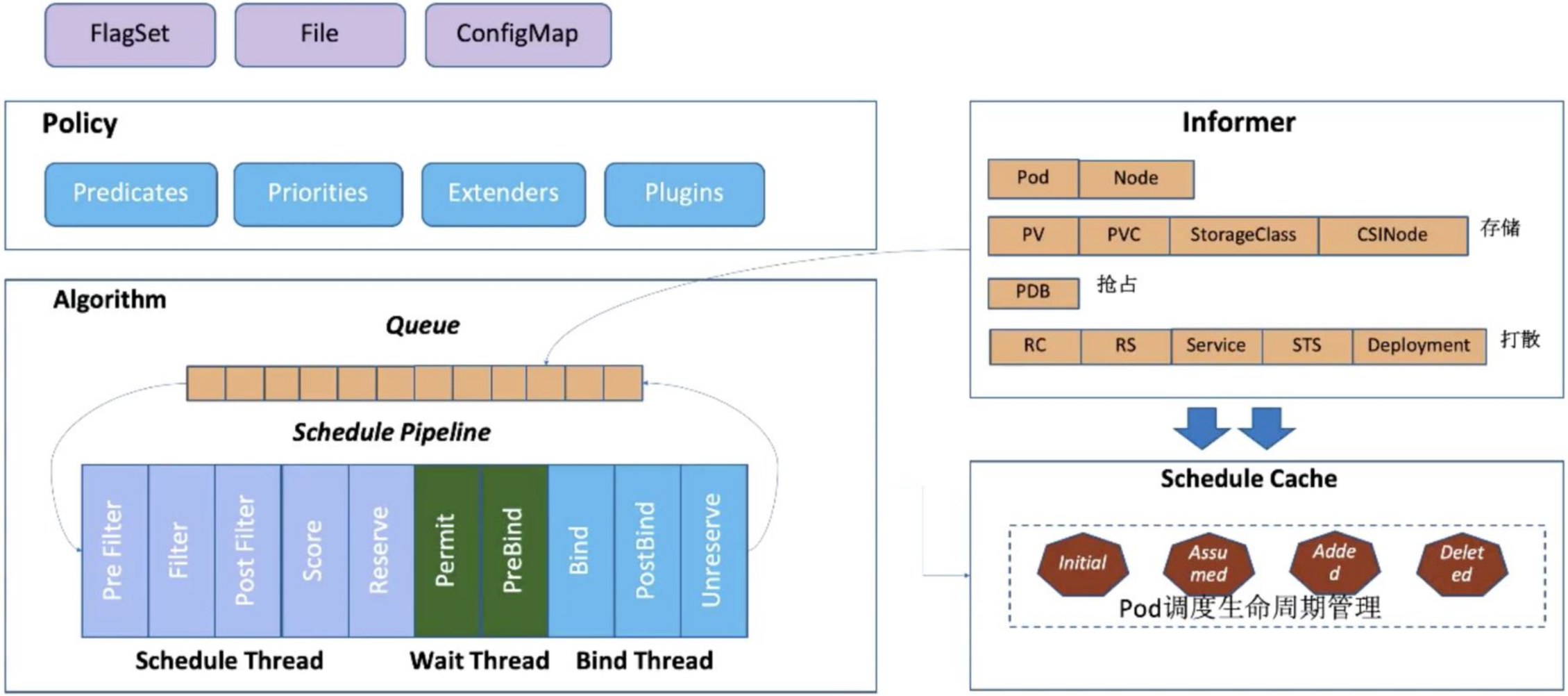
```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high
value: 10000
globalDefault: false
```

创建一个名为low的priorityClass，得分为100：

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: low
value: 100
globalDefault: false
```

为Pod1配置上high、Pod2配置上low priorityClassName：

```
spec:
  priorityClassName: high       # 或者 low
```


FlagSet
File
ConfigMap
Policy
Predicates
Priorities
Extenders
Plugins
Algorithm
Queue
Schedule Pipeline
Pre Filter
Filter
Post Filter
Score
Reserve
Permit
PreBind
Bind
PostBind
Unreserve
Schedule Thread
Wait Thread
Bind Thread
Informer
Pod
Node
PV
PVC
StorageClass
CSINode
存储
PDB
抢占
RC
RS
Service
STS
Deployment
打散
Schedule Cache
Initial
Assumed
Added
Deleted
Pod调度生命周期管理

如何部署GPU Kubernetes

1.安装 NVIDIA 驱动

```
$ sudo yum install -y gcc kernel-devel-$(uname -r)
$ sudo /bin/sh ./NVIDIA-Linux-x86_64*.run
```

2.安装 NVIDIA Docker2

```
$ sudo yum install nvidia-docker2
$ sudo pkill -SIGHUP dockerd
```

3.部署Nvidia Device Plugin

```
$ kubectl create -f nvidia-device-plugin.yml
```

```
bash# cat daemon.json
{
    "default-runtime": "nvidia",
    "runtimes": {
        "nvidia": {
            "path": "/usr/bin/nvidia-container-runtime",
            "runtimeArgs": []
        }
    }
}
```

daemon.json

```
bash# kubectl get po -l component=nvidia-device-plugin
NAME                                          READY     STATUS
nvidia-device-plugin-cn-shanghai.192.168.0.85   1/1       Running
```

验证部署GPU Kubernetes结果

```
$ kubectl describe node gpu-node-01

apiVersion: v1
kind: Node
metadata:
      name: node-1
...
Status:
      Capacity:
            cpu:                4
          memory:               15234152Ki
          nvidia.com/gpu:       2
```

```
apiVersion: v1
kind: Pod
metadata:
    name: cuda-vector-add
spec:
    restartPolicy: OnFailure
    containers:
        - name: cuda-vector-add
          image: "nvidia/cuda-vector-add:v0.1"
resources:
            limits:
              nvidia.com/gpu: 1
              memory: "512Mi"
             cpu: "250m"
```

通过nvidia.com/gpu指定
使用GPU的数量

```
$ kubectl apply -f gpupod.yaml
pod/gpu created

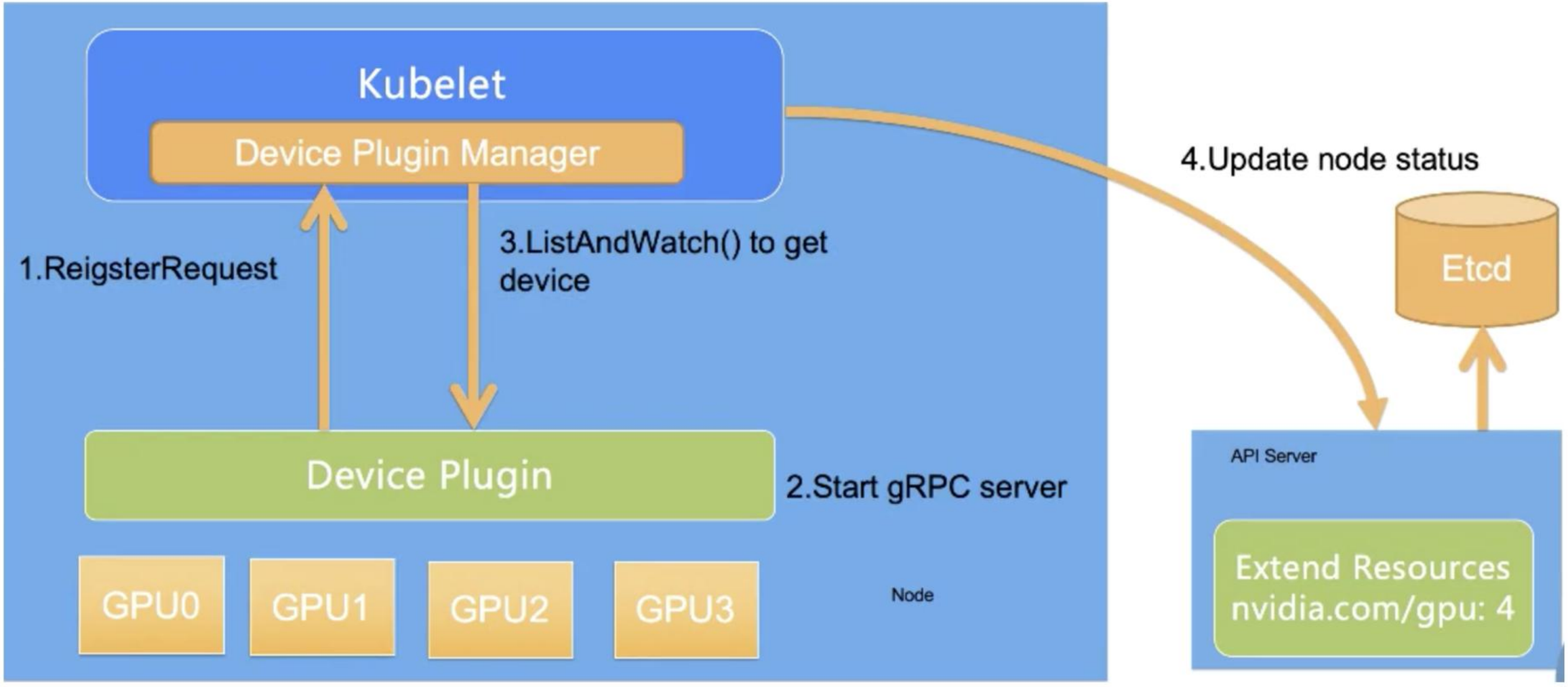
$ kubectl exec -it gpu -- nvidia-smi
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 410.79       Driver Version: 410.79       CUDA Version: 10.1     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|===============================+======================+======================|
|   0  Tesla T4            On   | 00000000:00:07.0 Off |                    0 |
| N/A   33C    P8    15W /  70W |      0MiB / 15079MiB |      0%      Default |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                       GPU Memory |
|  GPU       PID   Type   Process name                             Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
```

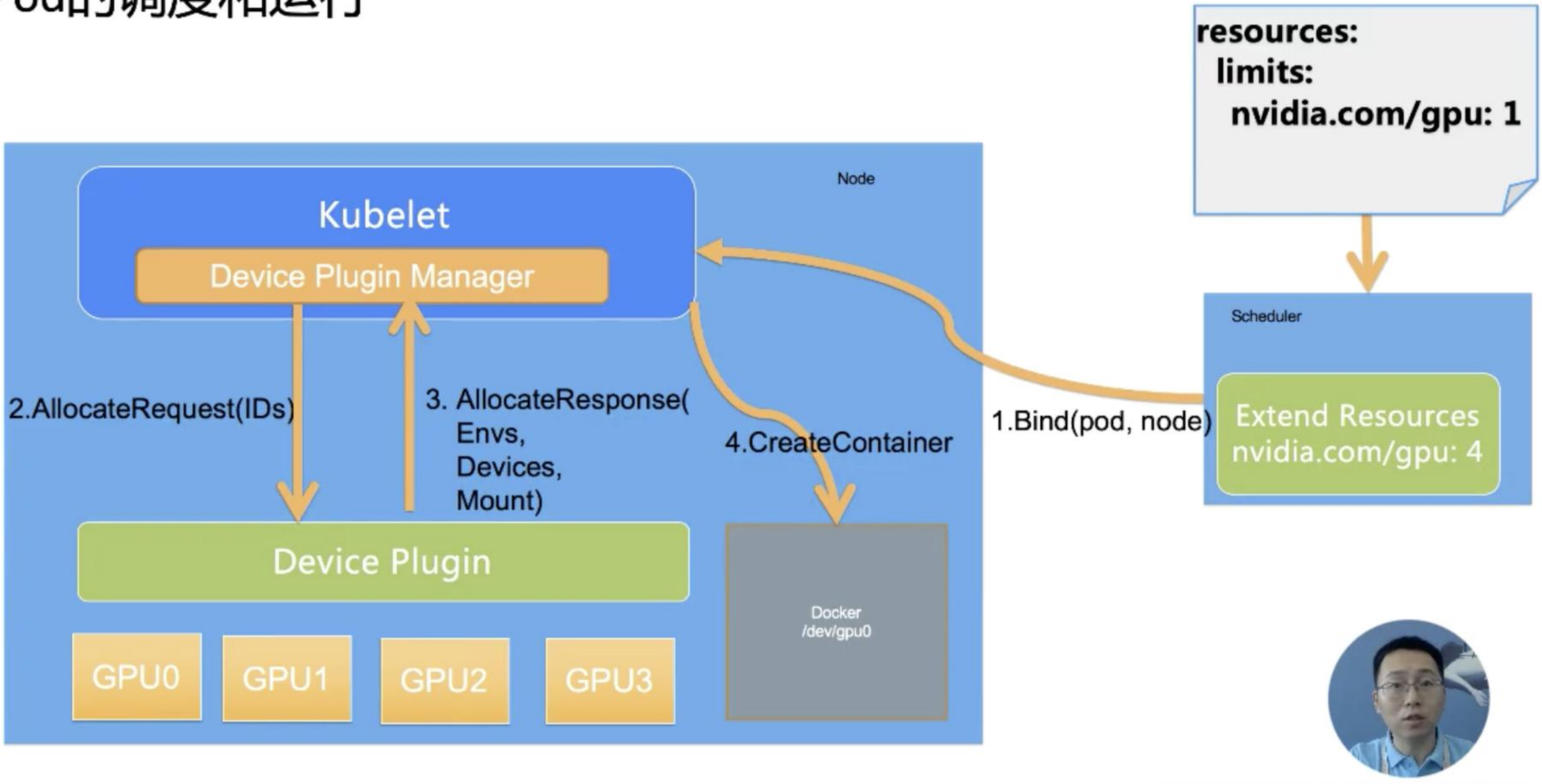
```
# 执行 PACTH 操作
$ curl --header "Content-Type: application/json-patch+json" \
--request PATCH \
--data '[{"op": "add", "path": "/status/capacity/nvidia.com/gpu", "value": "1"}]' \
https://localhost:6443/api/v1/nodes/<your-node-name>/status
```



```
$ kubectl describe node <your-node-name>
apiVersion: v1
kind: Node
...
Status:
      Capacity:
            nvidia.com/gpu:     1
```


Kubelet
Device Plugin Manager
1.ReigsterRequest
3.ListAndWatch() to get device
4.Update node status
Etcd
Device Plugin
2.Start gRPC server
API Server
GPU0
GPU1
GPU2
GPU3
Node
Extend Resources
nvidia.com/gpu: 4

Pod的调度和运行


resources:
limits:
nvidia.com/gpu: 1
Node
Kubelet
Device Plugin Manager
2.AllocateRequest(IDs)
3. AllocateResponse(
Envs,
Devices,
Mount)
4.CreateContainer
1.Bind(pod, node)
Scheduler
Extend Resources
nvidia.com/gpu: 4
Device Plugin
Docker
/dev/gpu0
GPU0
GPU1
GPU2
GPU3

社区的异构资源调度方案

- Nvidia GPU Device Plugin：https://github.com/NVIDIA/k8s-device-plugin
- GPU Share Device Plugin ：

  https://github.com/AliyunContainerService/gpushare-scheduler-extender

  https://github.com/AliyunContainerService/gpushare-device-plugin
- RDMA Device Plugin ：https://github.com/Mellanox/k8s-rdma-sriov-dev-plugin
- FPGA Device Plugin ：https://github.com/Xilinx/FPGA_as_a_Service/tree/master/k8s-fpga-device-plugin/trunk

绑定条件

VolumeMode 检查	筛选所有具有相同VolumeMode的volume；Block/FileSystem；
LabelSelector 检查	若PVC配置了selector,matchLabels，会筛选符合Selector条件的Volume；
StorageClassName 检查	如果配置了StorageClassName，会筛选value值相同的Volume；
AccessMode 检查	筛选PVC的AccessMode列表中所有选项，都有相应的AccessMode配置的volume；
Size 检查	筛选哪些大于等于PVC的Size 的volume，若数量大于1，则筛选size最小的volume；

```
$ kubectl get pod -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP              NODEGATES
nginx-web-0   1/1     Running   0          14m   172.27.0.133    cn-hangzhou.192.168.1.85
nginx-web-1   1/1     Running   0          14m   172.27.0.5      cn-hangzhou.192.168.1.83
nginx-web-2   1/1     Running   0          13m   172.27.1.9      cn-hangzhou.192.168.1.84
```

```
$ kubectl get pvc
NAME                      STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS              AGE
www-storage-nginx-web-0   Bound    disk-451c1393-ee56-11e9-a93f-be6d52dd44ce   20Gi
RWO            alicloud-disk-available   16m
www-storage-nginx-web-1   Bound    disk-4dd85adf-ee56-11e9-a93f-be6d52dd44ce   20Gi
RWO            alicloud-disk-available   16m
www-storage-nginx-web-2   Bound    disk-5bbf8820-ee56-11e9-a93f-be6d52dd44ce   20Gi
RWO            alicloud-disk-available   15m
```

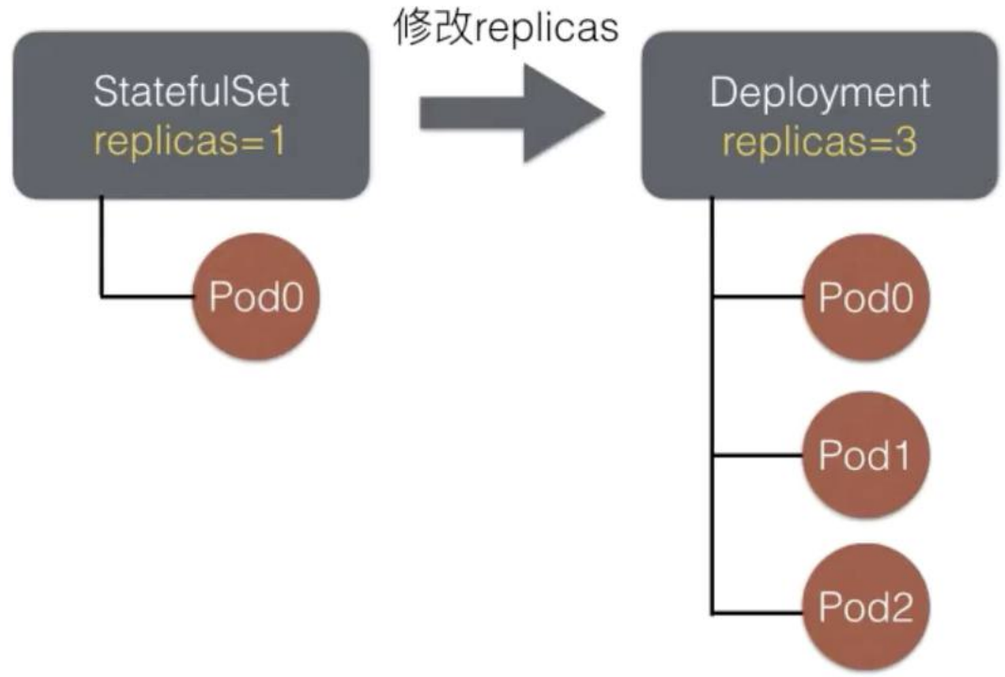


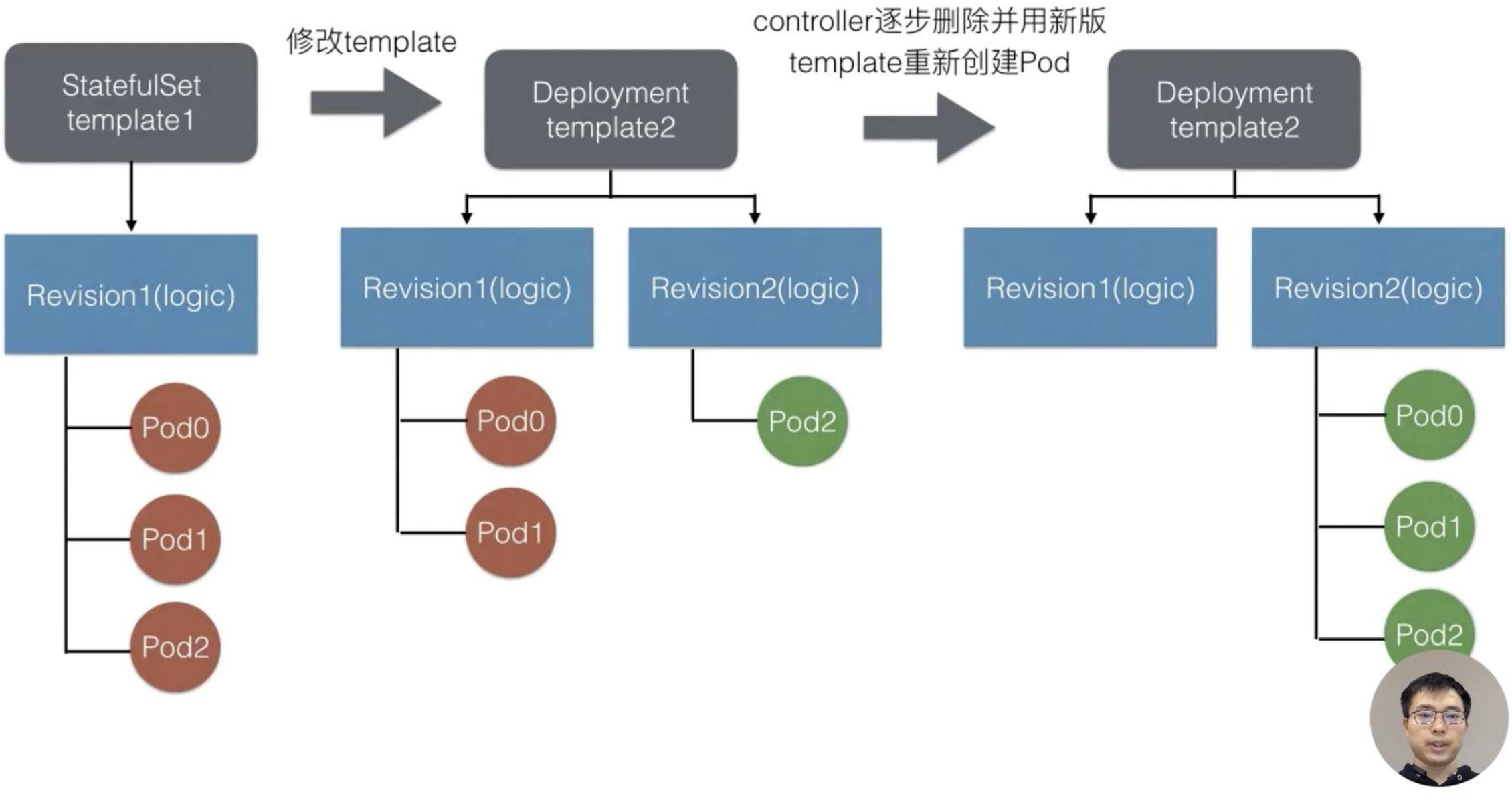

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
 generation: 2
  name: nginx-web
spec:
    # . . .
    spec:
      containers:
      - name: nginx
        image: nginx:mainline
```

StatefulSet.spec中，有一个字段名为podManagementPolicy，可选策略为OrderedReady和Parallel，默认为前者。

- OrderedReady：扩缩容按照order顺序执行。扩容时，必须前面序号的Pod都ready了，才能扩下一个；缩容时，按照倒序删除。

- Parallel：并行扩缩容，不需要等前面Pod都ready或删除后再处理下一个




StatefulSet
template1
修改template
Deployment
template2
controller逐步删除并用新版
template重新创建Pod
Deployment
template2
Revision1(logic)
Revision1(logic)
Revision2(logic)
Revision1(logic)
Revision2(logic)
Pod0
Pod1
Pod2
Pod0
Pod1
Pod2
Pod0
Pod1
Pod2

-

Replicas：期望数量

Selector：选择器，必须匹配.spec.template.metadata.labels

Template：Pod模板

VolumeClaimTemplates：PVC模板列表

```go
// A StatefulSetSpec is the specification of a StatefulSet.
type StatefulSetSpec struct {
    // replicas is the desired number of replicas of the given Template.
    // These are replicas in the sense that they are instantiations of the
    // same Template, but individual replicas also have a consistent identity.
    // If unspecified, defaults to 1.
    // TODO: Consider a rename of this field.
    // +optional
    Replicas *int32 `json:"replicas,omitempty" protobuf:"varint,1,opt,name=replicas"`

    // selector is a label query over pods that should match the replica count.
    // It must match the pod template's labels.
    // More info: https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/#label-selectors
    Selector *metav1.LabelSelector `json:"selector" protobuf:"bytes,2,opt,name=selector"`

    // template is the object that describes the pod that will be created if
    // insufficient replicas are detected. Each pod stamped out by the StatefulSet
    // will fulfill this Template, but have a unique identity from the rest
    // of the StatefulSet.
    Template v1.PodTemplateSpec `json:"template" protobuf:"bytes,3,opt,name=template"`

    // volumeClaimTemplates is a list of claims that pods are allowed to reference.
    // The StatefulSet controller is responsible for mapping network identities to
    // claims in a way that maintains the identity of a pod. Every claim in
    // this list must have at least one matching (by name) volumeMount in one
    // container in the template. A claim in this list takes precedence over
    // any volumes in the template, with the same name.
    // TODO: Define the behavior if a claim already exists with the same name.
    // +optional
    VolumeClaimTemplates []v1.PersistentVolumeClaim `json:"volumeClaimTemplates,omitempty" proto
```



ServiceName：对应Headless Service的名字，用于给Pod生成唯一网络标识

PodManagementPolicy：Pod管理策略

UpdateStrategy：Pod升级策略

RevisionHistoryLimit：保留历史ControllerRevision的数量限制（默认为10）

```
    // serviceName is the name of the service that governs this StatefulSet.
    // This service must exist before the StatefulSet, and is responsible for
    // the network identity of the set. Pods get DNS/hostnames that follow the
    // pattern: pod-specific-string.serviceName.default.svc.cluster.local
    // where "pod-specific-string" is managed by the StatefulSet controller.
    ServiceName string `json:"serviceName" protobuf:"bytes,5,opt,name=serviceName"`

    // podManagementPolicy controls how pods are created during initial scale up,
    // when replacing pods on nodes, or when scaling down. The default policy is
    // `OrderedReady`, where pods are created in increasing order (pod-0, then
    // pod-1, etc) and the controller will wait until each pod is ready before
    // continuing. When scaling down, the pods are removed in the opposite order.
    // The alternative policy is `Parallel` which will create pods in parallel
    // to match the desired scale without waiting, and on scale down will delete
    // all pods at once.
    // +optional
    PodManagementPolicy PodManagementPolicyType `json:"podManagementPolicy,omitempty" protobuf:"bytes,6,opt,name=podManag

    // updateStrategy indicates the StatefulSetUpdateStrategy that will be
    // employed to update Pods in the StatefulSet when a revision is made to
    // Template.
    UpdateStrategy StatefulSetUpdateStrategy `json:"updateStrategy,omitempty" protobuf:"bytes,7,opt,name=updateStrategy"

    // revisionHistoryLimit is the maximum number of revisions that will
    // be maintained in the StatefulSet's revision history. The revision history
    // consists of all revisions not represented by a currently applied
    // StatefulSetSpec version. The default value is 10.
    RevisionHistoryLimit *int32 `json:"revisionHistoryLimit,omitempty" protobuf:"varint,8,opt,name=revisionHistoryLimit"
```